

Python - Default Arguments

Python Default Arguments

Python allows to **define a function** with **default value** assigned to one or more formal arguments. Python uses the default value for such an argument if no value is passed to it. If any value is passed, the default value is overridden with the actual value passed.

Default arguments in Python are the function arguments that will be used if no arguments are passed to the function call.

Example of Default Arguments

The following example shows use of Python default arguments. Here, the second call to the function will not pass value to "**city**" argument, hence its default value "**Hyderabad**" will be used.

```
# Function definition
def showinfo( name, city = "Hyderabad" ):
    "This prints a passed info into this function"
    print ("Name:", name)
    print ("City:", city)
    return

# Now call showinfo function
```

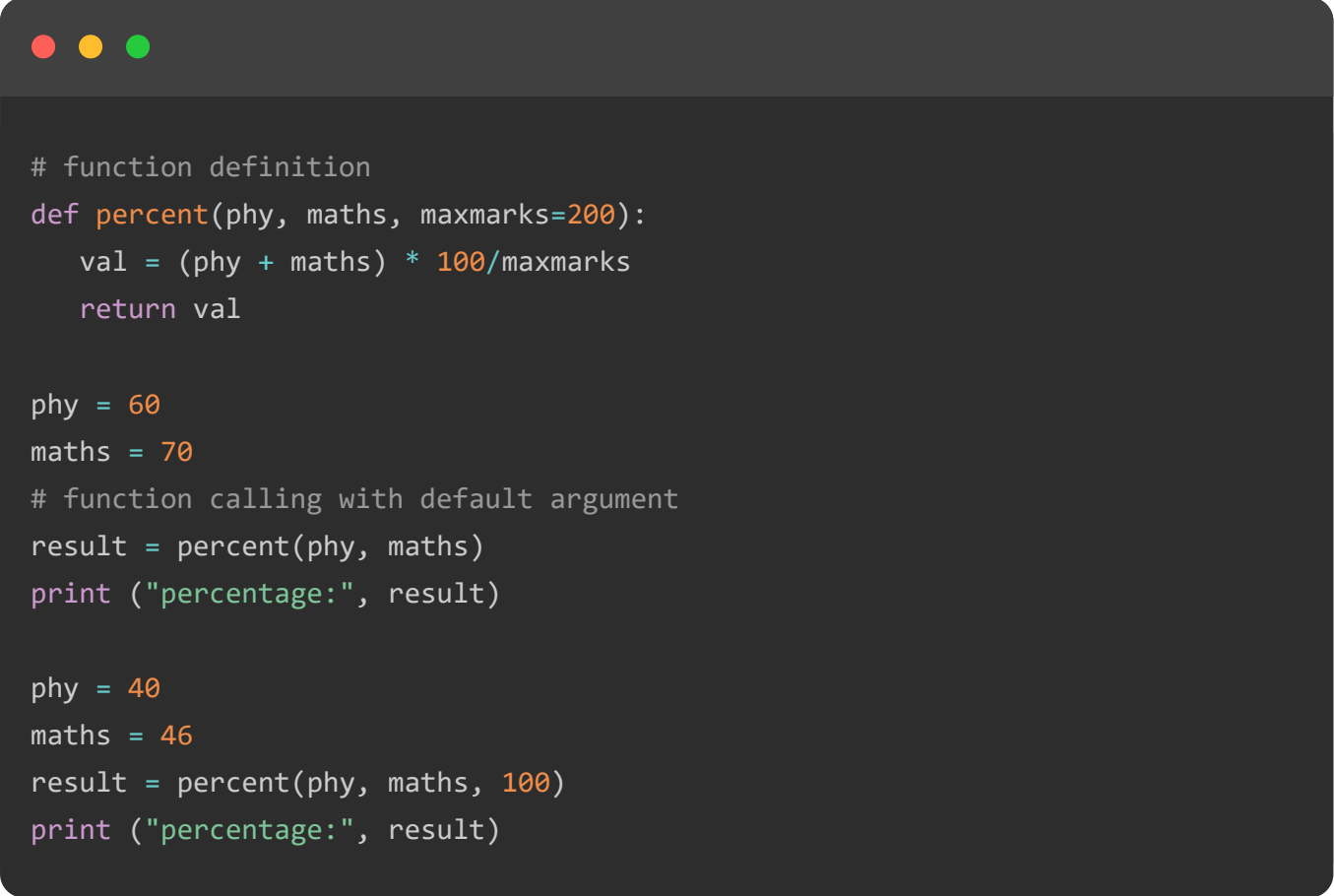
```
showinfo(name = "Ansh", city = "Delhi")
showinfo(name = "Shrey")
```

It will produce the following **output** –

```
Name: Ansh
City: Delhi
Name: Shrey
City: Hyderabad
```

Example: Calling Function Without Keyword Arguments

Let us look at another example that assigns default value to a function argument. The function **percent()** has a default argument named "**maxmarks**" which is set to 200. Hence, we can omit the value of third argument while calling the function.



```
# function definition
def percent(phy, maths, maxmarks=200):
    val = (phy + maths) * 100/maxmarks
    return val

phy = 60
maths = 70
# function calling with default argument
result = percent(phy, maths)
print ("percentage:", result)

phy = 40
maths = 46
result = percent(phy, maths, 100)
print ("percentage:", result)
```

On executing, this code will produce the following output –

```
percentage: 65.0
percentage: 86.0
```

Mutable Objects as Default Argument

Python evaluates default arguments once when the function is defined, not each time the function is called. Therefore, If you use a mutable default argument and modify it within the given function, the same values are referenced in the subsequent function calls.

Those Python objects that can be changed after creation are called as mutable objects.

Example

The code below explains how to use mutable objects as default argument in Python.

```
def fcn(nums, numericlist = []):  
    numericlist.append(nums + 1)  
    print(numericlist)  
  
# function calls  
fcn(66)  
fcn(68)  
fcn(70)
```

On executing the above code, it will produce the following output –

```
[67]  
[67, 69]  
[67, 69, 71]
```

TOP TUTORIALS

[Python Tutorial](#)

[Java Tutorial](#)

[C++ Tutorial](#)

[C Programming Tutorial](#)

C# Tutorial
PHP Tutorial
R Tutorial
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial



Chapters ▾

Categories ≡

Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler

[Online Python Compiler](#)
[Online Go Compiler](#)
[Online C Compiler](#)
[Online C++ Compiler](#)
[Online C# Compiler](#)
[Online PHP Compiler](#)
[Online MATLAB Compiler](#)
[Online Bash Terminal](#)
[Online SQL Compiler](#)
[Online Html Editor](#)

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.